Topic 3: Dictionaries

#eda #apunts #english #PARCIAL1

1. Abstract Data Types (ADT) & Dictionaries

- **Abstract Data Type (ADT)**: An ADT formalizes a data structure by specifying its **values**, **operations**, and the **properties** governing those operations, independent of implementation details.
- **Dictionary Definition**: A dictionary is a mathematical set of elements composed of pairs `(key, information)`, where keys belong to a domain K and are **unique**.
- **Core Dictionary Operations**:
  - **assign(key, info)**: Inserts a new key-information pair or overwrites existing information if the key is already present.
  - **delete(key)**: Removes the pair associated with the given key.
  - **consult(key) / search(key) / present(key)**: Checks whether a key exists and returns its associated information or reference.
  - **size()**: Returns the total number of key-value pairs stored.

---

2. Linear Implementations & Cost Comparison

When using basic linear structures (vectors or linked lists), operations require element shifting or sequential searching.

Structure	`assign` (Worst / Avg)	`delete` (Worst / Avg)	`consult` (Worst / Avg)
Non-sorted Vector	$\Theta(n)$ / $\Theta(n)$	$\Theta(n)$ / $\Theta(n)$	$\Theta(n)$ / $\Theta(n)$
Non-sorted List	$\Theta(n)$ / $\Theta(n)$	$\Theta(n)$ / $\Theta(n)$	$\Theta(n)$ / $\Theta(n)$
Sorted Vector	$\Theta(n)$ / $\Theta(n)$	$\Theta(n)$ / $\Theta(n)$	$\Theta(\log n)$ / $\Theta(\log n)$
Sorted List	$\Theta(n)$ / $\Theta(n)$	$\Theta(n)$ / $\Theta(n)$	$\Theta(n)$ / $\Theta(n)$
Hash Table	$\Theta(n)$ / $\Theta(1)$	$\Theta(n)$ / $\Theta(1)$	$\Theta(n)$ / $\Theta(1)$

---

3. Hash Tables

Hash tables generalise vectors by mapping keys to table slots via a **hash function** $h : K \to \{0, 1, \ldots, m-1\}$, where m is table size.

Direct-Access vs. Hash Tables

- **Direct-Access Tables**: Used when keys are small natural numbers; slot index equals key value ($T[k]$), giving $\Theta(1)$ worst-case time but wasting memory if key space K is huge.
- **Collisions**: A collision occurs when two distinct keys hash to the same slot ($h(k_1) = h(k_2)$). If $|K| > \alpha m$, at least $\alpha + 1$ keys are guaranteed to collide in some slot.

Collision Resolution Strategies

1. **Separate Chaining**:
   - Each table entry $T[i]$ holds a pointer to a linked list containing all elements mapped to slot i.
   - **Load Factor (α)**: Defined as $\alpha = n/m$ (average elements per bucket).
   - **Cost**:
     - **Worst-case**: $\Theta(n)$ when all keys map to a single list.
     - **Average-case**: $\Theta(1 + \alpha)$ for search, assign, and delete operations.
2. **Open Addressing**:
   - All elements are stored directly inside the table array. Collisions are resolved by probing alternative slots using a sequence $(h(k, 0), h(k, 1), \ldots, h(k, m - 1))$.

Hash Function Methods

- **Division Method**:
  - Formula: $h(k) = k \bmod m$.
  - **Design Rule**: Choose m to be a **prime number** not close to a power of 2 to ensure even key distribution.
- **Multiplication Method**:
  - Formula: $h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor = \lfloor m \cdot (k \cdot A - \lfloor k \cdot A \rfloor) \rfloor$, where $0 < A < 1$.
  - **Design Rule**: m is typically chosen as a power of 2 for fast bit-shifting. Knuth suggests $A \approx \frac{\sqrt{5}-1}{2} \approx 0.6180339887$.

---

4. Binary Search Trees (BST)

Definition & Properties

- **BST Property**: A binary tree where every node's key is **strictly greater** than all keys in its left subtree and **strictly smaller** than all keys in its right subtree.
- **In-Order Traversal**: Performing an in-order traversal on a BST visits elements in **strictly increasing sorted order**.

Complexity Analysis

- Operations (`search`, `assign`, `delete`, `minimum`, `maximum`) run in time proportional to tree height h: $\Theta(h)$.
- **Average Case**: For randomly built trees, expected height is $h = \Theta(\log n)$, giving $\Theta(\log n)$ **average time**.
- **Worst Case**: If insertions occur in sorted order, the tree degrades into a linear linked list of height $h = n$, leading to $\Theta(n)$ **worst-case time**.

---

5. Self-Balancing AVL Trees (Adelson-Velskii & Landis)

AVL Balance Property & Height Bound

- **Balance Condition**: An AVL tree is a BST where, for every node, the height difference between its left and right subtrees is **at most 1** ($|h_{\text{left}} - h_{\text{right}}| \leq 1$).
- **Guaranteed Height**: An AVL tree with n nodes satisfies $h = \Theta(\log n)$ in the worst case.
  - *Proof Idea*: The minimum number of nodes $T(h)$ in an AVL tree of height h satisfies $T(h) = T(h-1) + T(h-2) + 1 \geq (\sqrt{2})^h \implies h \leq \log_{\sqrt{2}}(n) = \Theta(\log n)$.

Rebalancing via Rotations ($\Theta(1)$ Cost)

When an `assign` or `delete` operation produces an imbalance factor of 2, balance is restored using pointer rotations in $\Theta(1)$ time:

1. **LL Case (Left-Left)**: Insertion into the left child's left subtree $\implies$ **Single Right Rotation** on root node.
2. **RR Case (Right-Right)**: Insertion into the right child's right subtree $\implies$ **Single Left Rotation** on root node.
3. **LR Case (Left-Right)**: Insertion into the left child's right subtree $\implies$ **Double Rotation**: Left rotation on left child, followed by Right rotation on root.
4. **RL Case (Right-Left)**: Insertion into the right child's left subtree $\implies$ **Double Rotation**: Right rotation on right child, followed by Left rotation on root.

Worst-Case Guarantees

- `consult`, `assign`, and `delete` all achieve $\Theta(\log n)$ **worst-case time** in AVL trees.